Conception détaillée

TIC315 — Projet 4, jeu d'échecs en ligne. Jalon J4.

Ce document explique comment le modèle du domaine (J2, J3) devient un modèle de conception, directement traduisible en Java au J5. Il justifie chaque décision : les patrons de conception, la répartition des responsabilités (GRASP), le respect des principes SOLID et le découpage en packages. Les contraintes OCL sont dans `contraintes-ocl.md`.

Diagrammes de conception :

Diagramme	Contenu
`packages-architecture`	Les couches et leurs dépendances
`classes-conception-partie`	Package `metier.partie` : parties, états (patron État), observateurs
`classes-conception-jeu`	Package `metier.jeu` : plateau, cases, pièces, coups
`classes-conception-joueur`	Package `metier.joueur` : joueurs, classements (patron Stratégie)
`classes-conception-controle`	Packages `ihm`, `controle`, `infrastructure` : MVC, adaptateurs, dépôts
`sequence-conception-jouer-coup`	Les patrons à l'œuvre pendant un coup

1. Du modèle du domaine au modèle de conception

Le modèle du domaine décrit les concepts du jeu. La conception ajoute ce qu'il faut pour coder : des types Java, un sens de navigation pour chaque association, des visibilités, et quelques classes techniques. Chaque écart est voulu :

Dans le domaine	Dans la conception	Raison
Types `Date`, `Duree`	`Instant`, `Duration` ; l'heure vient d'une `Clock`	Types standard de Java ; une horloge remplaçable rend la pendule testable en JUnit
Association Partie — Joueur dans les deux sens	Partie → Joueur seulement ; Joueur garde un indicateur `occupe`	Casser le cycle entre `metier.partie` et `metier.joueur` (section 2) ; `peutJouer()` n'a pas à parcourir l'historique
Classe-association Participation	Classe ordinaire : Partie compose ses participations, chacune désigne son joueur	Règle de traduction du cours (§6.2)
Association qualifiée Joueur [categorie] — Classement	`Map<CategorieCadence, Classement>`	Traduction directe d'un qualificatif

Dans le domaine	Dans la conception	Raison
Énumération `EtatPartie` et attribut `etat`	Classes d'état (patron État) ; `etat()` renvoie toujours l'énumération	Le comportement change avec l'état (section 3.1) ; l'énumération reste le vocabulaire commun à l'OCL, au diagramme d'états et à l'affichage
`/trait`, `/aDejaBouge`	`trait()` calculé sur le nombre de coups ; `aDejaBouge()` calculé sur un compteur de déplacements	Un attribut dérivé devient une méthode calculée (§6.2) ; la pièce n'a pas à remonter jusqu'aux coups de la partie
`annuler()`	`annuler(joueur)` et `annulerParLeSysteme()`	Qui annule change les règles : le créateur ou l'invité (RG13), un participant en début de partie (RG16) ; le système quand le créateur est suspendu (RG8) ou que le moteur ne répond plus (RG15)
`jouerCoupOrdinateur()` et `jouerCoup()`	Même chemin dans l'état et dans `executerCoup()`, avec un `joueur` absent pour l'ordinateur	Un coup de l'ordinateur subit la même vérification et le même enregistrement (UC7), mais l'ordinateur n'est pas un `Joueur`
Pendule et proposition de nulle, dans la séquence de `jouerCoup`	`apresCoup()` et `demarrer()`, qui n'agissent que dans `PartieEntreJoueurs`	Partie ne connaît pas la pendule (RG10) : polymorphisme plutôt qu'un test sur la sorte de partie
`deplacementsPossibles(plateau)`	`deplacementsPossibles(plateau, depart)`	C'est la case qui connaît sa pièce ; la pièce reçoit donc sa case de départ
Énumération `MotifFin`	Dans `metier.jeu`	`Plateau::situationFinale()` la renvoie ; ailleurs, elle créerait un cycle
Événement `when(temps du trait = 0)`	`verifierTemps()`, appelée par le contrôleur avant chaque action et par une tâche périodique (J5)	Un événement de changement n'est pas un appel : il faut quelqu'un pour le constater
`Joueur::suspendre()` avec les effets de RG8	`Joueur::suspendre()` change l'état du compte ; `ControleurCompte::suspendre()` retrouve la partie active du joueur et l'abandonne ou l'annule	Le joueur ne connaît plus ses parties ; la postcondition `plusDePartieActive` (C15) est garantie par le contrôleur

Les accesseurs simples ne sont pas dessinés : le cours ne les prévoit que « si nécessaire » (§6.2), et ils n'ajoutent aucune décision.

2. Packages

Package	Contenu	Dépend de
`ihm`	`VuePartie`	`controle`, `metier.partie`, `metier.jeu`
`controle`	`ControleurPartie`, `ControleurCompte` ; interfaces `MoteurEchecs`, `ServiceAuthentification`, `DepotParties`, `DepotJoueurs`	`metier`
`metier.partie`	`Partie` et ses sous-classes, `Participation`, `Cadence`, les états, `ObservateurPartie`, `EtatPartie`, `Resultat`	`metier.jeu`, `metier.joueur`
`metier.jeu`	`Plateau`, `Case`, `Piece` et ses sous-classes, `Coup`, `Couleur`, `TypePromotion`, `MotifFin`	aucun
`metier.joueur`	`Joueur`, `Classement`, `CalculClassement`, `CalculElo`, `EtatCompte`, `CategorieCadence`	aucun
`infrastructure`	`AdaptateurStockfish`, `AdaptateurOpenIdConnect`, `DepotPartiesSql`, `DepotJoueursSql`	`controle`, `metier`

Les règles du cours (§2.3.2) sont respectées :

- **Aucun cycle.** Le seul cycle possible, entre `partie` et `joueur`, est cassé par la navigabilité Partie → Joueur ; celui entre `partie` et `jeu`, par le déplacement de `MotifFin`.
- **Dépendre de ce qui est stable.** Le métier ne dépend ni de l'interface, ni de la base, ni des services externes.
- **Inversion des dépendances.** `controle` définit les interfaces dont il a besoin ; `infrastructure` les implémente. Remplacer Stockfish ou la base ne touche ni le métier ni les contrôleurs.
- **Pas de package fourre-tout.** Chaque classe est rangée selon son rôle.

3. Patrons de conception

Le cours le rappelle : « pas de patron pour le plaisir ». Chaque patron ci-dessous répond à un problème précis du projet.

3.1 État — le comportement de la partie dépend de son état

Problème. Neuf événements (rejoindre, refuser l'invitation, annuler, jouer un coup, abandonner, proposer, accepter ou refuser la nulle, temps écoulé) n'ont pas le même effet selon que la partie est en attente, en cours, terminée ou annulée. Coder cela dans Partie mettrait un `switch` sur l'état au début de chaque méthode, et chaque nouvel état obligerait à modifier toutes ces méthodes.

Solution. Une classe abstraite `EtatDePartie` déclare une opération par événement du diagramme d'états ; par défaut, chacune refuse en levant `TransitionInterditeException`. Chaque état concret (`EtatEnAttente`, `EtatEnCours`, `EtatTerminee`, `EtatAnnulee`) ne redéfinit que les événements qu'il accepte. Partie délègue l'événement à son état courant ; l'état fait le travail en appelant Partie, puis change l'état si la transition le demande (`changerEtat`).

Où. `classes-conception-partie`, `sequence-conception-jouer-coup` (messages 4 à 9).

Conséquences. Le diagramme d'états se lit directement dans le code : une classe par état, une méthode par transition. `verifier.py` contrôle que chaque événement du diagramme d'états a son opération dans `EtatDePartie`. États terminaux : `EtatTerminee` et `EtatAnnulee` n'acceptent rien. Le prix : cinq classes de plus, et des opérations de Partie en visibilité package pour que les états puissent agir sur elle.

Principes. Ouvert/fermé (un nouvel état est une nouvelle classe), responsabilité unique (chaque état connaît ses seules transitions), polymorphisme (GRASP).

3.2 Observateur, avec MVC — prévenir les deux écrans

Problème. Quand un joueur joue, l'écran de son adversaire doit se mettre à jour, comme ceux des deux joueurs quand la partie démarre, se termine ou reçoit une proposition de nulle. La séquence du J3 se termine d'ailleurs par « notifier : nouveau plateau » sans dire qui le fait. Si Partie appelait directement la couche web, le métier dépendrait de l'interface.

Solution. Partie tient une liste d'`ObservateurPartie` (interface du métier) et les notifie après chaque changement. `VuePartie`, dans `ihm`, réalise cette interface ; chaque joueur connecté a la sienne, abonnée à sa partie. Modèle « tiré » : la vue reçoit la partie et relit ce qui l'intéresse, ce qui garde une seule opération `actualiser(partie)` au lieu d'une par événement.

MVC. La vue transmet les actions au contrôleur, qui les délègue au modèle ; le modèle prévient la vue par l'Observateur, sans la connaître (cours, §5.4.6).

Où. `classes-conception-partie`, `classes-conception-controle`, `sequence-conception-jouer-coup` (messages 19 à 21).

Conséquences. Faible couplage entre le métier et l'interface. Attention, comme le dit le cours, aux observateurs oubliés : la vue d'un joueur qui quitte la partie doit se désabonner (`desabonner`).

3.3 Adaptateur — parler à Stockfish et au service d'authentification

Problème. Stockfish s'utilise par le protocole UCI (des commandes texte sur l'entrée standard d'un processus) ; le service d'authentification, par OpenID Connect. Aucune de ces interfaces ne doit apparaître dans les contrôleurs.

Solution. `controle` définit les interfaces dont il a besoin, dans son vocabulaire : `MoteurEchecs::calculerCoup(position, force)` et `ServiceAuthentification::verifierIdentite(jeton)`. `AdaptateurStockfish` et `AdaptateurOpenIdConnect`, dans `infrastructure`, les traduisent dans le protocole réel.

Où. `classes-conception-controle`.

Conséquences. On peut remplacer Stockfish par un autre moteur UCI, ou le fournisseur d'identité, sans toucher au reste. Les tests du contrôleur utilisent un faux moteur. Un moteur muet se traduit par `MoteurIndisponibleException`, qui mène à l'annulation de la partie (RG15).

3.4 Stratégie — la formule du classement

Problème. Le sujet dit que le classement évolue « selon la formule choisie » : la formule est un point de variation annoncé. Si elle était codée dans Partie, en changer obligerait à modifier la classe la plus centrale du modèle.

Solution. `PartieEntreJoueurs` délègue le calcul à une `CalculClassement`, reçue à sa création. `CalculElo` est l'implémentation retenue (section 6). La partie contre l'ordinateur ne

calcule rien : `mettreAJourClassements()` n'est redéfinie que dans `PartieEntreJoueurs` (polymorphisme plutôt qu'un test sur la sorte de partie).

Où. `classes-conception-joueur`, `sequence-conception-jouer-coup` (messages 11 à 17).

Conséquences. Changer de formule, par exemple pour Glicko, utilisé par lichess et chess.com, revient à ajouter une classe (ouvert/fermé). La formule se teste seule, sans partie.

Limite assumée. Une seule formule est implémentée aujourd'hui. Le cours conseille d'introduire un patron « quand la rigidité apparaît » : ce patron se justifie ici par une variation annoncée par le sujet (GRASP Protection des variations), pas par deux variantes déjà présentes. C'est le moins fort des quatre patrons ; les patrons État et Observateur suffisent à eux seuls à l'exigence de deux patrons justifiés.

3.5 Patrons écartés

Patron	Pourquoi il n'est pas retenu
Singleton (pour le moteur)	Il cacherait la dépendance au moteur et empêcherait de le remplacer dans les tests ; le moteur est passé au contrôleur
Fabrique (pour les pièces)	Les pièces ne sont créées qu'à deux endroits connus : la position initiale du plateau et la promotion. Aucune variation à cacher
Composite	Aucune hiérarchie tout-partie récursive dans le domaine
Décorateur	Un seul canal de notification (la vue web) ; rien à cumuler
Commande	Servirait à annuler un coup ; reprendre un coup est hors périmètre

4. Répartition des responsabilités (GRASP)

Patron GRASP	Application
Expert en information	Le plateau vérifie la légalité d'un coup (il connaît les cases) ; chaque pièce calcule ses déplacements ; la partie vérifie le trait et les nulles automatiques (elle connaît les coups) ; la participation gère sa pendule (elle connaît son temps)
Créateur	La partie crée ses coups, ses participations et son plateau : elle les contient (composition)
Contrôleur	`ControleurPartie` et `ControleurCompte` reçoivent les actions des joueurs et délèguent, sans faire le travail
Polymorphisme	Déplacements par sorte de pièce ; comportement par état ; pendule et classements selon la sorte de partie
Fabrication pure	Les dépôts (`DepotParties`, `DepotJoueurs`) gardent la persistance hors du métier
Indirection	Le contrôleur entre la vue et le métier ; les interfaces entre le contrôleur et les services externes
Protection des variations	`CalculClassement`, `MoteurEchecs`, `ServiceAuthentification`
Faible couplage, forte cohésion	Packages sans cycle ; `metier.jeu` ne connaît ni joueur ni partie

5. Principes SOLID

Principe	Respect dans la conception
Responsabilité unique	La partie gère le déroulement, le plateau les règles de déplacement, la participation la pendule, le calcul la formule, le dépôt la persistance
Ouvert/fermé	Un nouvel état, une nouvelle formule de classement ou un nouveau moteur s'ajoutent par une nouvelle classe
Substitution de Liskov	Toute sous-classe de Partie, de Piece ou d'EtatDePartie s'utilise là où la classe mère est attendue ; un état qui n'accepte pas un événement le refuse par la même exception que la classe mère
Ségrégation des interfaces	Interfaces courtes et ciblées : `ObservateurPartie` (une opération), `CalculClassement` (une), `MoteurEchecs` (une), `ServiceAuthentification` (une) ; un dépôt par sorte d'objet
Inversion des dépendances	Les contrôleurs dépendent d'interfaces, implémentées par l'infrastructure ; le métier possède `ObservateurPartie`, réalisée par l'interface utilisateur

6. Formule du classement (RG6)

Formule retenue : **Elo**, la formule de la FIDE.

- Score attendu d'un joueur de `points` contre un adversaire de `pointsAdversaire` : `1 / (1 + 10^((pointsAdversaire − points) / 400))`.
- Variation : `arrondi(K × (score − scoreAttendu))`, avec un score de 1 pour une victoire, 0,5 pour une nulle et 0 pour une défaite.
- **K = 20**, le coefficient de la FIDE pour les parties rapides et blitz. Chaque joueur commence à **1500** points dans chaque famille de cadence.
- Seul le classement de la famille de la cadence de la partie change ; les parties contre l'ordinateur et les parties annulées ne changent rien (RG6, RG15).

Exemple, qui servira de test JUnit au J5 : un joueur à 1500 bat un joueur à 1480. Le gagnant attendait 0,53 : il gagne 9 points, et le perdant en perd 9. Sur une nulle, le mieux classé perd 1 point et l'autre en gagne 1.

L'arrondi peut rendre la somme des deux variations différente de zéro d'un point ; c'est accepté, comme sur la plupart des sites. **Glicko** a été écarté pour le code : il demande de suivre en plus la fiabilité de chaque classement, et le patron Stratégie permet de l'ajouter plus tard.

7. Des contraintes OCL et des refus au code (J5)

Les règles écrites en OCL deviennent des vérifications explicites (cours, §6.2) :

Dans le modèle	Dans le code Java
Invariants (C1 à C8)	Vérifiés dans les constructeurs et les opérations qui modifient l'objet ; l'unicité (C1) par le dépôt, qui voit tous les joueurs
Préconditions (C12, C13, C14, C15)	Exceptions au début de l'opération : `TransitionInterditeException` si l'état refuse l'événement, `CoupRefuseException` si ce n'est pas le trait du joueur ou si le coup est illégal
Postconditions	Assertions des tests JUnit
Attributs dérivés (C9, C10)	Méthodes calculées `trait()` et `aDejaBouge()`

Dans le modèle	Dans le code Java
Requête `peutJouer()` (C11)	Méthode de Joueur, sur son état et l'indicateur `occupe`

Les tests JUnit du J5 suivront ces règles : un test par règle de gestion, au moins pour RG1 à RG6, RG11, RG13 et RG16. Ce tableau prépare la matrice de traçabilité du dossier final (cas d'utilisation, puis classes, puis tests).

8. Points à valider en équipe

1. **Stratégie de classement.** La garder comme quatrième patron, avec sa limite écrite (section 3.4), ou la présenter seulement comme une application de GRASP Protection des variations.
2. **Indicateur** `occupe`**.** Il double une information calculable depuis les parties du joueur. Toute partie le libère dans `terminer()` ou `cloreSansResultat()` : à couvrir par des tests au J5.
3. **K = 20 et 1500 points au départ.** Valeurs de la FIDE, faciles à changer (constructeur de `CalculElo`, constante de `Classement`).